BCT Practical Guide: BCT2, BCT3, BCT4

Exam prep for Blockchain Technology. Covers the exact steps to carry out each practical, code explanations, and viva questions.

1. Difficulty Ranking

1. **BCT2 (MetaMask wallet): easiest.** No code. Only steps. The risk is faucet delay, so get test ETH tonight.
2. **BCT3 (Bank contract): medium.** Three simple functions. Learn the deposit, withdraw, balance flow and the wei units.
3. **BCT4 (Student data): medium.** Struct, array, fallback. Learn how to trigger the fallback and where to read gas and fee.

If you learn all three, you are covered for every slip.

2. Tonight's Checklist

- MetaMask installed in the browser and the wallet is created.
- Sepolia network visible in MetaMask.
- At least 0.05 Sepolia ETH in the wallet (more is better, because deploying costs gas).
- Remix opens: https://remix.ethereum.org
- Practice each practical once, start to finish, without looking at the steps.
- Keep `Bank.sol` and `StudentData.sol` ready to copy-paste.
- Note your wallet address and keep a friend's address handy for a transfer demo.
- Do not share your Secret Recovery Phrase with anyone. Keep it written down for your own recovery.

3. BCT2: Create a Wallet Using MetaMask for Crypto Transactions

Aim: Create a MetaMask wallet, get test ETH on the Sepolia test network, and make a transaction.

Steps

1. **Install MetaMask.** Open the Chrome or Edge extension store (or metamask.io), search MetaMask, click Add to Chrome, then pin the fox icon to the toolbar.
2. **Create a wallet.** Open MetaMask, click Create a new wallet, agree to the terms, and set a password (this password only works on this device).
3. **Secure the wallet.** MetaMask shows a 12-word Secret Recovery Phrase. Write it down in order and confirm it when asked. Anyone with this phrase controls your funds.
4. **Wallet is created.** You now have an account (Account 1) with a public address starting with `0x`. Click the address to copy it.
5. **Show test networks.** Click the network dropdown at the top, then turn on Show test networks (or in Settings, then Advanced, turn on Show test networks).
6. **Switch to Sepolia.** Pick Sepolia from the network list. Balance shows 0 SepoliaETH.
7. **Get test ETH from a faucet.** Copy your address and use one of these:
   - **Google Cloud Web3 Faucet:** sign in with a Google account, paste address, request.
   - **Alchemy Sepolia Faucet:** about 0.1 ETH per day per wallet. It may ask you to sign in and may have a mainnet balance requirement.
   - **Chainlink Faucet:** connect wallet, sign in, choose Sepolia, request.
   - **PoW Faucet (pk910):** no login needed. You "mine" in the browser for a while and then claim. Good fallback if the others reject you.
   - **Backup plan:** ask a classmate to send you a little Sepolia ETH.
   - Faucet rules change often, so try 2 or 3 and do it before the exam day if possible.
8. **Check balance.** After a minute or two, MetaMask shows the SepoliaETH balance.
9. **Send a transaction.** Click Send, paste the receiver address (friend's or your second account), enter a small amount (for example 0.001), click Next, review the gas fee, and click Confirm.
10. **Verify.** Open the Activity tab in MetaMask. The status goes from Pending to Confirmed. Click it, then View on block explorer (Sepolia Etherscan) to see the transaction hash, from, to, value, gas used, and fee.
11. **Optional.** Click the three dots, then Account details, and show the QR/address. Create a second account (Add account) to show a transfer between your own accounts.

What to show the examiner

The wallet address, the Sepolia balance, one completed transaction, and the transaction hash on Etherscan.

4. BCT3: Bank Smart Contract (Deposit, Withdraw, Show Balance)

Aim: Write a smart contract on a test network for a customer's bank account with deposit, withdraw, and show-balance operations.

Steps (in order)

1. Open **https://remix.ethereum.org**.
2. In the File Explorer, create a new file called `Bank.sol` and paste the code from the attached `Bank.sol`.
3. Go to the **Solidity Compiler** tab. Choose a compiler version 0.8.x (same family as `pragma solidity ^0.8.0`). Click **Compile Bank.sol**. A green tick appears on the tab.
4. Go to the **Deploy & Run Transactions** tab.
5. Set **Environment** to **Injected Provider - MetaMask**. MetaMask pops up, so connect your account and make sure the network is **Sepolia**.
6. Confirm the contract selected is **Bank**, then click **Deploy**.
7. MetaMask shows a confirmation with the gas fee. Click **Confirm**. Wait for the green tick in the Remix console.
8. Under **Deployed Contracts**, expand the contract. Copy the contract address if needed.
9. **Deposit:** In the **Value** box type `1000000000000000` and keep the unit as **Wei** (this is 0.001 ETH). Or type `0.001` and change the unit to **Ether**. Click the orange **deposit** button and confirm in MetaMask.
10. **Show balance:** Click the blue **showBalance** button. It returns the balance in wei, so `1000000000000000` means 0.001 ETH.
11. **Withdraw:** Reset the Value box to `0`. Type `500000000000000` in the withdraw box (this is 0.0005 ETH) and click **withdraw**. Confirm in MetaMask.
12. **Show balance again:** Click **showBalance**. It should now show `500000000000000`.
13. **Verify on Etherscan:** Click the transaction in the Remix console and open it on Sepolia Etherscan, or open the MetaMask activity to show each transaction.

Code explanation

Part	Meaning
`// SPDX-License-Identifier: MIT`	License line. Solidity gives a warning if it is missing.
`pragma solidity ^0.8.0;`	Compiler version 0.8.0 or higher (below 0.9.0).
`contract Bank { }`	Like a class. It holds the data and functions.
`mapping(address => uint256) private balances;`	A key-value store. Each customer address maps to their balance in wei.
`event Deposited / Withdrawn`	Logs that appear in the transaction details. They are cheap and good for history.
`function deposit() public payable`	`payable` lets the function receive ETH.
`msg.sender`	The address of whoever calls the function.
`msg.value`	The ETH (in wei) sent with the call.
`require(condition, "message")`	Stops and reverts the transaction if the condition fails.
`balances[msg.sender] -= amount;`	Reduces the balance before sending money. This is the safe order.
`call{value: amount}("")`	Sends ETH to the caller. Returns `sent` as true or false.
`function showBalance() public view`	`view` means read-only. It costs no gas when called directly.

Units to remember

- 1 ETH = 10^18 wei
- 1 Gwei = 10^9 wei
- 0.001 ETH = 10^15 wei = `1000000000000000`

Improvements over the original code

- Added `Deposited` and `Withdrawn` events so each action is logged.

- Added `require(amount > 0)` in withdraw.
- Added short comments for explaining in the viva. The logic is unchanged and stays simple.

5. BCT4: Student Data Contract (Structure, Array, Fallback) with Gas and Fee

Aim: Create a Solidity program for student data using structures, arrays, and a fallback function. Deploy on Ethereum (Sepolia test network) and observe the transaction fee and gas values.

Steps (in order)

1. Open **https://remix.ethereum.org**.
2. Create `StudentData.sol` and paste the code from the attached file.
3. **Solidity Compiler** tab: select 0.8.x, click **Compile StudentData.sol**.
4. **Deploy & Run Transactions** tab: Environment = **Injected Provider - MetaMask**, network = **Sepolia**.
5. Select **StudentData** and click **Deploy**. Confirm in MetaMask. **Note the gas fee shown here.**
6. **Add a student:** Under the deployed contract, expand **addStudent**. Enter for example `1`, `"Riya"`, `20`, `"Computer Engineering"`. Click **transact**, then confirm in MetaMask.
7. **Add one more student** with different values (for example `2`, `"Aman"`, `21`, `"IT"`).
8. **Read data:** Click **getTotalStudents** (should show 2). Enter `0` in **getStudent** to see the first student. You can also use the public `students` getter with an index.
9. **Test the fallback function.** At the bottom of the deployed contract, find **Low level interactions**.
   - To trigger `fallback()`: type random data in **Calldata**, for example `0x1234`, and click **Transact**. No function matches this data, so the fallback runs.
   - To trigger `receive()`: leave Calldata empty, set a Value (for example 100 Wei), and click **Transact**.
   - Open the transaction logs to see the `FallbackCalled` event.
10. **Observe gas and fee:**
    - In the **Remix console**, click the transaction to expand it. See `gas`, `transaction cost`, `execution cost`.

- In **MetaMask**, see the estimated gas fee before confirming.
- On **Sepolia Etherscan** (click the transaction hash), see **Gas Used**, **Gas Price**, **Gas Limit**, and **Transaction Fee**.

11. Note how deployment costs much more gas than `addStudent`, and a read call (`view`) costs nothing.

Gas formula

Transaction fee = Gas Used x Gas Price

Example with made-up numbers: 200,000 gas used x 2 Gwei = 400,000 Gwei = 0.0004 ETH.

Code explanation

Part	Meaning
`struct Student { id, name, age, course }`	A custom data type that groups related fields.
`Student[] public students;`	A dynamic array of structures. `public` makes Solidity create an automatic getter.
`students.push(Student(...))`	Adds a new student to the end of the array.
`string memory _name`	Strings need `memory` as the data location for function parameters.
`getStudent(index)`	Returns one student's details. `require` checks that the index is valid.
`getTotalStudents()`	Returns the array length.
`fallback() external payable`	Runs when a call matches no function. Also runs when data is sent that does not match.
`receive() external payable`	Runs when plain ETH is sent with empty calldata.
`emit FallbackCalled(...)`	Logs who called, the amount, and the data.

Improvements over the original code

- The original only had `receive()`. The practical asks for a **fallback**, so a proper `fallback()` was added next to `receive()`. It records `msg.data` so you can show what was sent.
- Kept comments short and the logic simple.

6. Quick Troubleshooting

Problem	Fix
Deploy button does nothing or MetaMask does not pop up	Click the MetaMask fox icon, check for a pending request, and reconnect via Environment dropdown.
Insufficient funds	Get more Sepolia ETH from a faucet.
Wrong network	Switch MetaMask to Sepolia, then redeploy.
Compile error about version	Pick a compiler 0.8.x in the Compiler tab.
Transaction stuck on Pending	Wait. If it takes too long, use Speed up in MetaMask.
Withdraw fails: Insufficient balance	You are withdrawing more than you deposited, or you are using a different account. Check with showBalance.
Withdraw amount confusion	The withdraw input is in wei, not ETH. Use `500000000000000` for 0.0005 ETH.
Value box not reset	Before calling withdraw or any non-payable function, set Value to 0.

7. Viva Questions and Short Answers

A. Very Basic Blockchain Questions

1. **What is a blockchain?** A distributed digital ledger where data is stored in linked blocks across many computers. Records cannot be easily changed.

2. **What is a block?** A group of transactions plus a timestamp, the previous block's hash, and its own hash.
3. **How are blocks linked?** Each block stores the hash of the previous block, forming a chain.
4. **What is a hash?** A fixed-length output from a function such as SHA-256. Any change in the input changes the hash completely.
5. **What is decentralization?** No single authority controls the network. Many nodes share and verify the data.
6. **What is a node?** A computer that participates in the blockchain network.
7. **What is a consensus mechanism?** The method nodes use to agree on valid transactions. Examples: Proof of Work and Proof of Stake.
8. **Proof of Work vs Proof of Stake?** PoW uses computing power (mining). PoS uses coins staked by validators. Ethereum now uses PoS.
9. **What is mining?** Solving a computational puzzle to add a block in PoW chains.
10. **What is a cryptocurrency?** A digital currency secured by cryptography, such as Bitcoin and Ether.
11. **What is a smart contract?** A program stored on the blockchain that runs automatically when conditions are met.
12. **What is Ethereum?** A blockchain platform that supports smart contracts and decentralized applications.
13. **What is a dApp?** A decentralized application that runs on a blockchain through smart contracts.
14. **What is a public vs private blockchain?** Public: anyone can join (Ethereum). Private: restricted access (enterprise).
15. **Why is blockchain secure?** Cryptographic hashing, digital signatures, decentralization, and consensus.
16. **What is immutability?** Once data is recorded and confirmed, it is practically impossible to change.
17. **What is a digital signature?** A value created with your private key to prove you authorized a transaction.
18. **What is a public key and private key?** Public key (address): shared with everyone. Private key: secret, used to sign.

B. BCT2: MetaMask and Wallet Questions

1. **What is a crypto wallet?** Software that stores your keys and lets you send, receive, and manage crypto. It does not actually hold coins; coins are on the blockchain.
2. **What is MetaMask?** A browser extension and mobile wallet to interact with Ethereum and dApps.
3. **What is a Secret Recovery Phrase?** A 12-word backup that can restore your wallet and all its accounts. Never share it.
4. **Difference between public address and private key?** The address is for receiving funds; the private key proves ownership and signs transactions.
5. **Custodial vs non-custodial wallet?** Non-custodial (MetaMask): you hold the keys. Custodial (an exchange): the company holds them.
6. **Hot wallet vs cold wallet?** Hot: connected to the internet (MetaMask). Cold: offline (hardware wallet).
7. **What is a testnet?** A practice blockchain using valueless test coins. Example: Sepolia.
8. **Why use Sepolia?** To test and deploy contracts for free without spending real money.
9. **What is a faucet?** A website that gives free test ETH.
10. **What is gas?** The fee paid for computation and storage on Ethereum.
11. **What is a transaction hash?** A unique ID of a transaction, used to track it on a block explorer.
12. **What is Etherscan?** A block explorer to view transactions, blocks, addresses, and contracts.
13. **Can test ETH be converted to real money?** No, it has no real value.
14. **What happens if you lose your recovery phrase?** You cannot recover the wallet. Nobody can reset it for you.

C. BCT3: Bank Contract Questions

1. **What does the contract do?** It lets each customer deposit ETH, withdraw it, and view their balance.
2. **What is `mapping`?** A key-value data structure. Here: address to balance.
3. **Why is `payable` used?** Only payable functions can receive ETH.
4. **What is `msg.sender` and `msg.value`?** Caller's address and the ETH sent with the call.
5. **What is `require`?** A check that reverts the transaction with an error message if the condition is false.

6. **What is `view`?** The function only reads data and does not change state, so it is free to call.
7. **What is wei?** The smallest unit of ETH. 1 ETH = 10^18 wei.
8. **Why do deposit and withdraw cost gas but showBalance does not?** They change the blockchain state. showBalance only reads.
9. **Why is the balance reduced before sending ETH?** To prevent reentrancy attacks, where a malicious contract calls withdraw again before the balance updates.
10. **What is an event?** A log entry emitted by the contract, useful for tracking activity.
11. **What happens if you withdraw more than your balance?** `require` fails, the transaction reverts, and the message "Insufficient balance" is shown.
12. **Why `private` for balances?** It can only be accessed inside this contract. Note that blockchain data is still technically readable by anyone.
13. **Can one customer see another's balance?** showBalance returns only the caller's balance, so each customer sees their own.
14. **What is ABI?** The interface (list of functions and inputs) that tools use to interact with the contract.
15. **What is bytecode?** The compiled code that is stored on the blockchain.

D. BCT4: Student Data Contract Questions

1. **What is a structure?** A user-defined type that groups different variables. Here: id, name, age, course.
2. **What is an array?** A list of items of the same type. Here: a dynamic array of Student structs.
3. **Static vs dynamic array?** Static has a fixed size. Dynamic can grow using `push`.
4. **What does `push` do?** Adds an element at the end of the array.
5. **What is `memory` in `string memory`?** A temporary data location used during a function call. Strings and structs in parameters need it.
6. **What is the difference between memory, storage, and calldata?** Storage: permanent on the blockchain. Memory: temporary during the call. Calldata: read-only input data.
7. **What is a fallback function?** A special function that runs when a call matches no function or when data is sent with no matching function.
8. **What is `receive()`?** A special function that runs when plain ETH is sent with empty calldata.

9. **fallback vs receive?** receive handles empty calldata with ETH. fallback handles non-matching calldata (and also ETH if there is no receive).
10. **How did you test the fallback?** Using Low level interactions in Remix, sending random calldata such as `0x1234`.
11. **What is `msg.data`?** The complete data (calldata) sent with the call.
12. **What is gas?** The unit measuring computation work on Ethereum.
13. **What is gas limit, gas used, gas price?** Limit: the maximum gas you allow. Used: the gas actually consumed. Price: what you pay per gas unit (in Gwei).
14. **How is the transaction fee calculated?** Gas Used x Gas Price.
15. **Why does deployment cost more gas than addStudent?** Deployment stores the whole contract code on the blockchain.
16. **Why does a longer name cost more gas?** More data to store.
17. **Why do view functions cost no gas?** They are read locally on a node and do not create a transaction.
18. **What is a base fee and priority fee?** Base fee is set by the network and burned. Priority fee (tip) goes to the validator.

E. Common to All Practicals

1. **What is Remix IDE?** A browser-based tool to write, compile, deploy, and test Solidity contracts.
2. **What is Solidity?** A programming language for writing smart contracts on Ethereum.
3. **What is EVM?** Ethereum Virtual Machine, which executes smart contract code on every node.
4. **What does Injected Provider - MetaMask mean?** Remix uses your MetaMask wallet to sign and send transactions to the selected network.
5. **What is the compile step?** Converts Solidity source into bytecode and ABI.
6. **What is a contract address?** The address on the blockchain where the deployed contract lives.
7. **Can deployed contracts be changed?** No, they are immutable. You deploy a new version instead.
8. **What does `pragma` mean?** It sets the compiler version for the file.
9. **What is the difference between a transaction and a call?** A transaction changes state and costs gas. A call only reads and is free.

10. **What is a revert?** The transaction is cancelled and its state changes are undone, but the gas used is still spent.
11. **Why use a testnet before mainnet?** It avoids real money loss from bugs.
12. **Name some real-world uses of blockchain.** Cryptocurrency, supply chain tracking, land records, healthcare records, voting, certificates.

8. Last-Minute Memory Aids

- **BCT2 flow:** Install, create wallet, save phrase, show test networks, Sepolia, faucet, send, Etherscan.
- **BCT3 flow:** Remix, paste, compile, MetaMask environment, deploy, deposit, balance, withdraw, balance.
- **BCT4 flow:** Remix, paste, compile, deploy, add students, get data, fallback with low-level calldata, gas and fee on Etherscan.
- **Fee formula:** Gas Used x Gas Price.
- **Units:** 1 ETH = 10^18 wei, 1 Gwei = 10^9 wei.
- **Before withdraw or reads:** set the Value box back to 0.